MORIROKU PHILIPPINES, INC. — INTERNAL IT

Form Request System

System & Architecture Documentation

Person Gate Pass • Material Gate Pass • Vehicle Scheduling • QR Security Scanning

Version 1.0 • July 2026

Live environment: http://192.168.9.7/FormRequestSystem/

Prepared by the IT Department

Contents

1 Introduction — what this system is

The **Form Request System** is an internal web application that replaces the paper gate pass process. Instead of filling out a paper form, walking it around for signatures, and hoping it does not get lost, an employee files the request on a computer or phone. Approvers review and e-sign it from their own dashboard, and the guard at the gate scans a QR code on the printed form to record the exact time a person or item leaves and returns.

The two forms it handles today

Form	Purpose	Special features
Person Gate Pass	An employee needs to leave the premises during work hours (official business or personal). May include companions and a company vehicle with a driver.	QR Time OUT and Time IN at the gate; vehicle and driver assignment by HRAD; trip types (drop-off, pick-up, or both).
Material Gate Pass	Items or materials need to be released through the gate (tools, documents, equipment). Lists up to 20 items with photos as proof.	Itemized list with quantities and units; photo proof required; printable letterhead form; items are released when the guard scans Time OUT.

What the system provides end to end

- **One login for everything** — employees sign in with their Employee ID; what they see depends on their role.
- **Automatic approval routing** — the system knows who must approve each request and in what order. Nobody can approve their own request.
- **Electronic signatures** — approvers draw or upload their signature once and reuse it; the signature appears on the printed form.
- **Daily control numbers** — every form receives a control number in the familiar office format, generated automatically without duplicates.
- **QR-verified gate activity** — the guard scans; the system decides whether it is a Time OUT or Time IN and records the exact timestamp. Manual entry of the control number is a fallback.
- **Vehicle and truck scheduling** — a calendar of company vehicle bookings and fixed weekly schedules, with Excel export matching the office monitoring workbooks.
- **Notifications** — requesters and approvers are notified inside the app when action is needed or a decision is made.
- **Complete audit trail** — every status change, scan, and admin action is recorded and cannot be edited.

> **Built to grow.** The system was deliberately designed as a general *form request platform*, not just a gate pass tool. New form types — for example a **Leave Form**, **Overtime (OT) Form**, or a general **Request Form** — can be added on top of the same login, approval, signature, notification, and audit machinery. Section 11 explains exactly how.

2 Users and roles

Access is role-based. An account can hold several roles at once; the interface adapts to the highest one. Roles and their permissions are stored in the database, so they can be reassigned without changing code.

Role	Who this usually is	What they can do
Associate (default)	Any regular employee	File Person and Material gate passes; track their own requests; print approved forms.
Immediate Superior	Section heads / managers	Everything an Associate can, plus approve or reject requests from their department with an e-signature.
HRAD / PAS Noter	Personnel & Admin Section staff	Final "noting" step of every request; assign vehicles, drivers, and schedule windows; manage fixed weekly schedules; place requests on hold; view department logs.
President	Company president	Approves company-vehicle requests and requests filed by superiors; can book a service vehicle directly; read-only department logs.
Security	Guards at the gate	Scanner screen and live gate queue only. Scan QR codes (or type the control number) to record Time OUT and Time IN with a guard signature.
Driver	Company drivers	Same as Associate; can also be assigned to vehicle bookings.
Truck Schedule Manager	Logistics / PPC staff	Manage recurring truck schedules on the calendar (trucks only).
System Administrator	IT	Manage users, roles, departments, vehicles, and drivers; view gate pass logs and the audit trail. Cannot scan (reserved for Security) and cannot approve.

A **public calendar** (no login needed) shows the driver and truck schedule so anyone in the office can check vehicle availability before filing a request.

3 How a request flows

3.1 Person Gate Pass

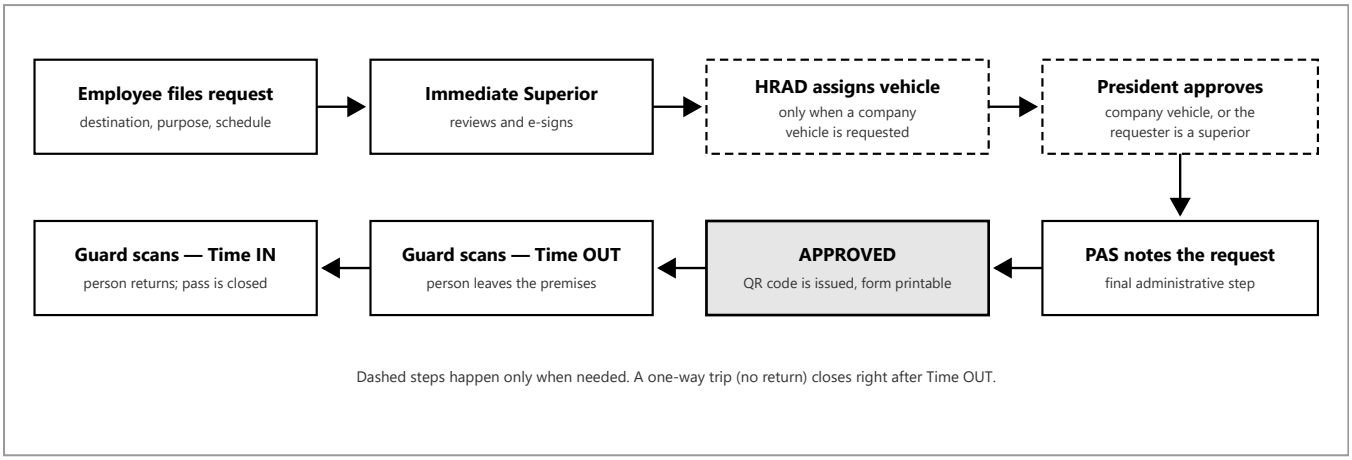

Figure 1 — Person Gate Pass: from filing to gate scanning

3.2 Material Gate Pass

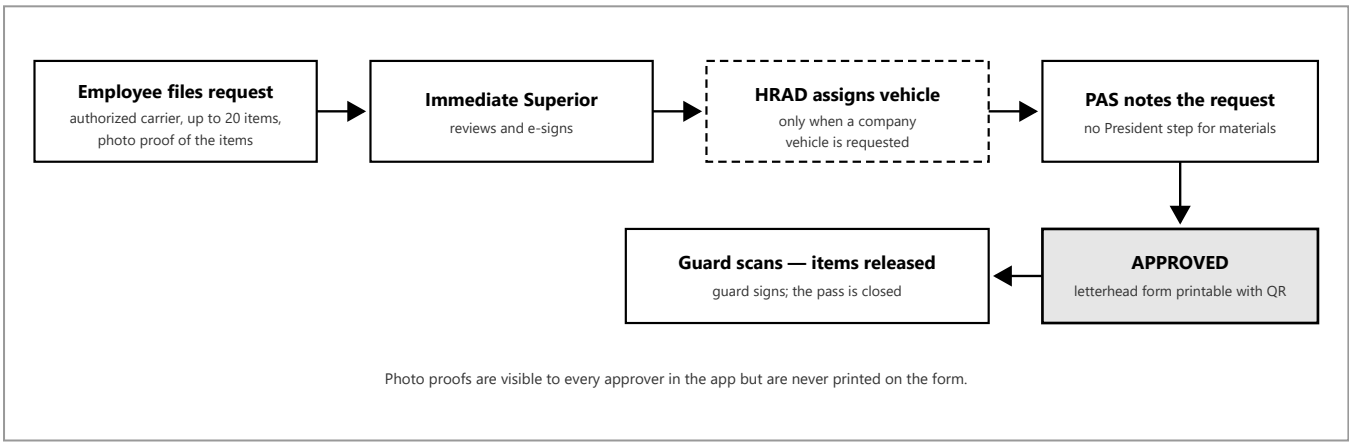

Figure 2 — Material Gate Pass: itemized release with photo proof

Built-in safeguards: the approval order is fixed when the request is submitted; a requester can never approve their own request; a rejection always requires a written reason; and every decision is stamped with the approver's e-signature, date, and time.

4 Request status lifecycle

Every request is always in exactly one status. The statuses are stored as data (not hard-coded), which keeps the list expandable. The guard's scanner only ever sees requests in the three "scannable" states.

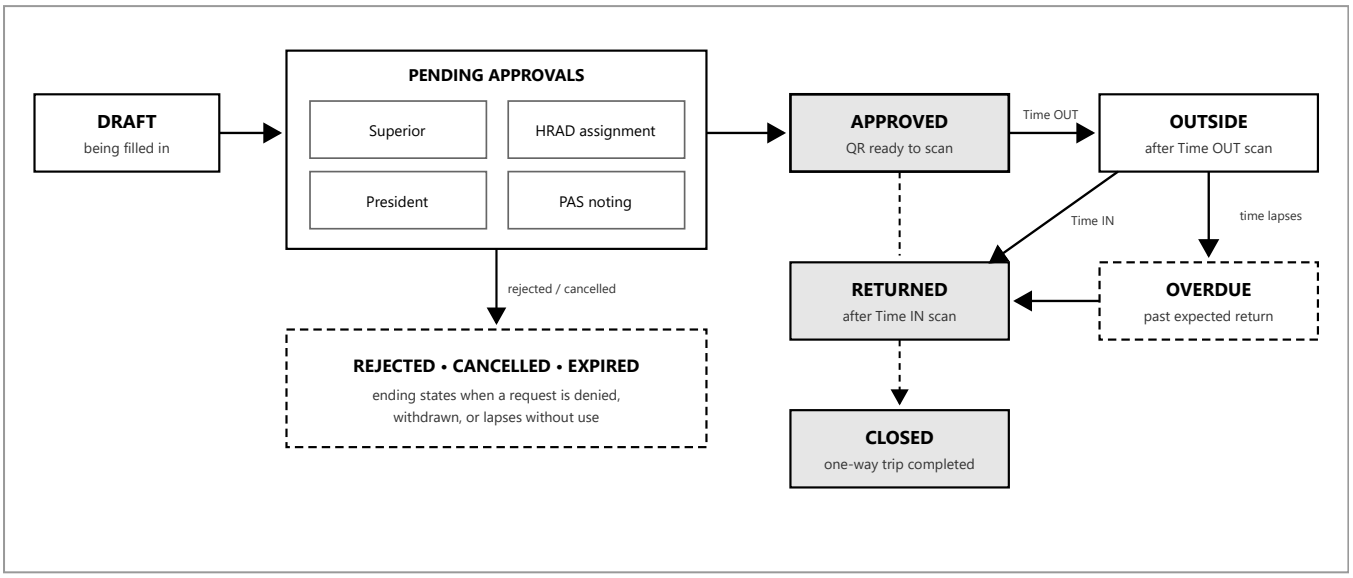

Figure 3 — Status lifecycle. Solid lines are scanner or approval actions; dashed lines are the one-way-trip shortcut.

Every transition writes an entry in an **immutable status history** table with who changed it and when. An "On Hold" state also exists so HRAD can pause a request (for example while waiting for a vehicle) without rejecting it.

5 System architecture

The system has three layers running on the office server **192.168.9.7**: the web frontend (plain HTML and JavaScript served by Apache), the API (ASP.NET Core 8 on port 5087), and the MariaDB database. Browsers never talk to the database directly — everything passes through the API, which enforces all the business rules.

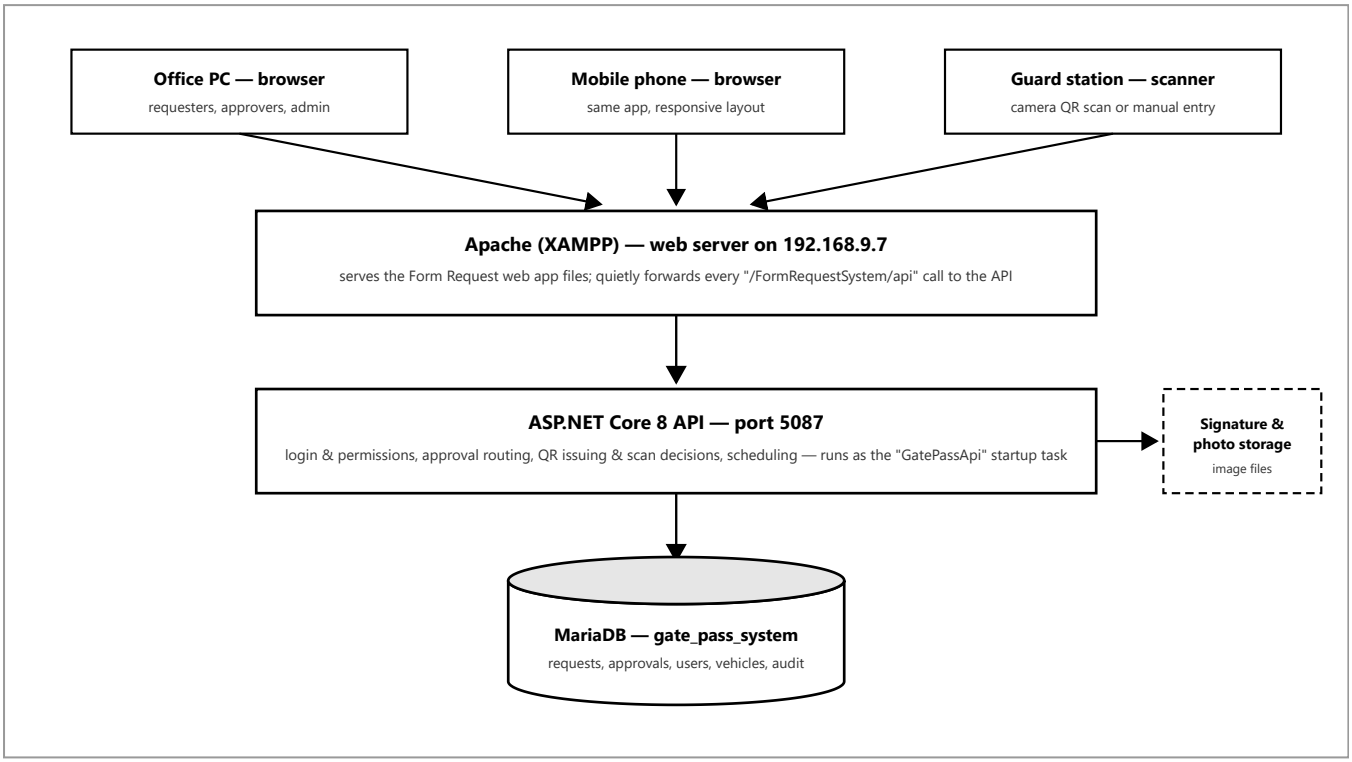

Figure 4 — Production architecture on the office server

5.1 Frontend

Plain HTML, CSS, and JavaScript — no framework and no build step. The single page `index.html` loads modules from `Frontend/Functions/` (one file per feature area: gate pass form, material form, approvals, scanner, calendar, admin, signatures, user manual). A small `runtime-config.js` detects where the app is running (developer PC, LAN, or the office server) and picks the right API address automatically — the same files run unchanged in every environment.

5.2 API

ASP.NET Core 8 with two projects: the web host (controllers, login, security policies) and a domain project (services, repositories, DTOs). Data access uses Dapper with stored procedures for the critical writes, so control numbers and status changes happen atomically inside the database. The API also enforces rate limits, response compression, security headers, and a uniform error format.

5.3 Database

MariaDB (bundled with XAMPP). The schema is versioned: a canonical `schema.sql` plus numbered migration files, all executed by one runner script that records every applied version in a ledger table. The same runner works on a developer PC and on the live server, which keeps environments identical.

5.4 Ways to run the system

Mode	What it is for
Office server (production)	The live system at 192.168.9.7 used by employees. Frontend under Apache, API as a startup task, MariaDB local to the server.
Developer PC (local)	Full stack on one machine for development and testing, with an option to expose it to phones on the same Wi-Fi.
Docker	A portable copy of the whole stack (web server, API, database) for demos or testing on any machine.
Temporary internet tunnel	Short-lived password-protected public link for off-network testing. **Not for production**, and it triggers endpoint-security warnings (see Section 9).

6 Database design

The database has around 40 tables, but the design is easy to hold in your head: **one central table holds the requests**, a ring of child tables holds everything that belongs to a request, and a layer of reference tables holds master data and the lists of allowed values.

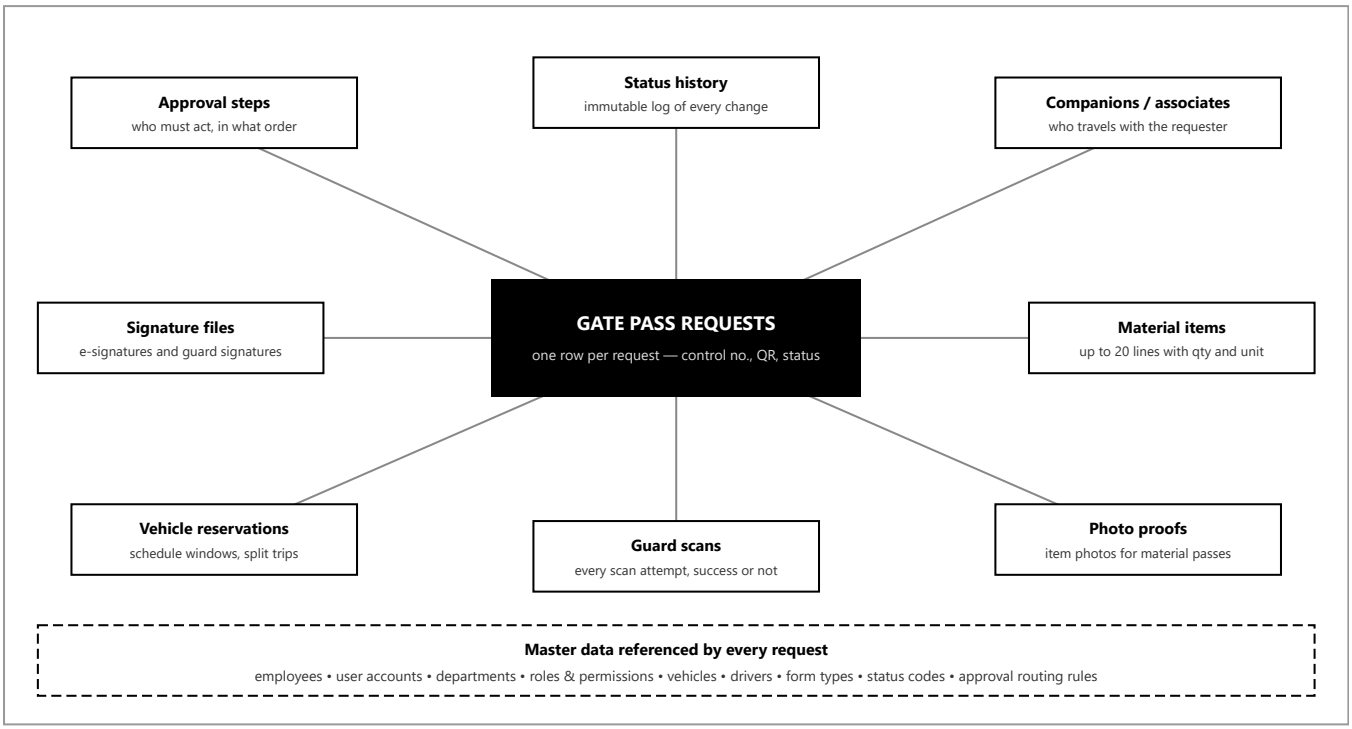

Figure 5 — The request table at the center, its child tables around it, master data underneath

6.1 Design principles that matter

- **Statuses and types are data, not code.** Form types, statuses, trip types, scan actions, and roles live in reference tables. Adding a new value is a seed row, not a schema change — this is what makes new form types cheap to add.
- **Approval routing is configuration.** A routing table maps each approval step and form type to the person responsible, with priorities, alternates, and validity dates. Changing an approver is a data change.
- **Nothing is deleted.** Requests reach a terminal status; vehicles and users are archived, not removed; the status history and audit logs are append-only.
- **Versioned migrations.** Every schema change is a numbered file recorded in a ledger, applied by one runner script (`Database/setup-xampp.ps1`) that is safe to re-run. Automatic backups are taken before risky steps.
- **Atomic writes.** Control-number generation and status transitions run inside stored procedures with transactions and row locks, so two people submitting at the same moment can never collide.

7 API overview

The API is the only door to the database. Every call (except login, the health probe, and public reference lists) requires a signed login token, and each endpoint additionally checks a specific permission. Responses use one consistent envelope with a trace ID for troubleshooting.

Group	Main endpoints	Purpose
Authentication	`POST /api/auth/login`, `GET /api/auth/me`	Sign in with Employee ID and password; returns the login token, roles, and permissions.
Requests	`POST /api/form-requests`, `POST /api/form-requests/material`, `GET /api/form-requests/my`, `GET /api/form-requests/{id}`, `GET /api/form-requests/{id}/qr`	Create Person and Material requests, list your own, open details, fetch the QR code.
Approvals	`GET /api/approvals/queue`, `POST /api/approvals/{id}/approve`, `POST /api/approvals/{id}/reject`	The approver's inbox and decisions; the approve call also carries HRAD's vehicle, driver, trip type, and schedule window.
Security	`GET /api/security/queue`, `POST /api/security/scans`, employee-pass lookups	Guard-only. The scan endpoint decides by itself whether a scan is a Time OUT or Time IN.
Fleet	`/api/vehicles`, `/api/drivers`, `/api/fleet/schedule`, fixed-schedule management	Vehicle and driver master lists (cached), the calendar feed, and recurring weekly schedules.
Admin	`/api/admin/users`, `/api/admin/roles`, `/api/admin/departments`	User, role, and department management; archiving instead of deleting.
Signatures	`POST /api/signatures`, `GET /api/signatures/{id}`	Upload and retrieve signature and proof images (size-limited, permission-checked).
Notifications	`GET /api/notifications/unread`, `POST /api/notifications/mark-read`	The bell icon; polled by the app every few seconds.
Dashboard & health	`GET /api/dashboard`, `GET /api/health`	Counters for the dashboard; a public probe that reports whether the API and database are up.

Protection layers on every request: login token check, then permission check, then rate limiting (a strict limit on login attempts, a general limit on everything else), then business-rule validation inside the service and the database procedures.

8 Security model

8.1 Sign-in and permissions

- Passwords are stored only as salted PBKDF2 hashes — nobody, including IT, can read a password from the database.
- A successful login issues a signed token valid for a limited time; the browser keeps it only for the session (closing the tab logs you out).
- Permissions ride inside the token, and every API endpoint declares exactly which permission it needs. The scanner is additionally locked to the Security role alone — even administrators cannot record gate scans.
- Login attempts are rate-limited to slow down password guessing; accounts can be locked and archived.

8.2 QR codes

- QR codes are cryptographically signed by the server — a QR made in a photo editor will not scan as valid.
- The database stores only a fingerprint of the QR, never the QR itself.
- The scan endpoint decides the action itself (Time OUT versus Time IN) based on the request's current status; a guard cannot record the wrong direction.
- A 30-second cooldown per pass prevents accidental double scans, and every attempt — valid or not — is recorded.

8.3 Auditability

- Status history, scan records, and audit logs are append-only, with the acting user, timestamp, and a trace ID.
- E-signatures are stored as files with checksums and linked to the specific approval step where they were applied.

8.4 Production guards built into the code

- The API refuses to start in production if the developer-only signing key is still configured.
- The database connection in production must come from a server environment variable, never from a file in the repository.
- Real employee data, passwords, and server credentials are kept out of the Git repository entirely (the `LocalData/` folder is ignored by Git).

9 Server operations — 192.168.9.7 (SSH-only runbook)

The live system runs on the office machine **n-timekeepingpc** (192.168.9.7). It also hosts the company's centralized dashboard, so the golden rule is: **touch only the Form Request folders and the `gate_pass_system` database — never Apache, MariaDB services, other folders, or any network settings.**

9.1 Why CrowdStrike Falcon warns, and how to work without triggering it

> **The warnings are expected behavior, not an infection.** CrowdStrike Falcon (the company antivirus) runs on both the office PCs and the server. It flags behavior that *looks like* attacker activity: batch files that launch hidden background processes, tunneling tools that open a path to the internet, and unsigned helper scripts. Several convenience scripts in this project do exactly those things, so Falcon reports them even though they are legitimate.

Avoid on office machines	Use instead
Launcher batch files that start the API in a hidden window (`Start_FormRequest_Backend*.bat` and similar)	The API on the server already starts by itself through the `GatePassApi` startup task — no launcher needed. For local development, run the API in a normal visible terminal.
The temporary internet tunnel script (`start-public.ps1` / cloudflared)	Test on the office network directly; the system is LAN-only by design.
Copying files to the server with ad-hoc helper scripts	Plain `ssh` and `scp` commands (below). These are Microsoft-signed Windows tools that Falcon trusts.

9.2 The SSH-only access pattern

All server work goes through the built-in Windows OpenSSH client using a key that stays on the IT workstation. No passwords are typed, nothing extra is installed, and every action is an ordinary visible command.

```
Connect to the server:
    ssh -i ~/.ssh/id_ed25519_gatepass User@192.168.9.7

Check that the live system is healthy (from any office PC):
    curl http://192.168.9.7:5087/api/health
    (expect: status Healthy, database Connected)

Copy updated frontend files to the server:
    scp -i ~/.ssh/id_ed25519_gatepass -r "path\to\files" "User@192.168.9.7:C:/xampp/htdocs/FormRequestSystem/"

Restart only the API (never Apache or MariaDB):
    ssh -i ~/.ssh/id_ed25519_gatepass User@192.168.9.7 "schtasks /end /tn GatePassApi"
    ssh -i ~/.ssh/id_ed25519_gatepass User@192.168.9.7 "schtasks /run /tn GatePassApi"
```

9.3 Deployment checklist (every time)

1. **Back up first** — copy the live frontend folder, the API folder, and a database dump into `C:\GatePassSystem\backups\` with a date-stamped folder name.
2. **Stage, then swap** — copy new files into a staging folder on the server, verify, then mirror into the live path.
3. **Database changes only via migration files** — add the numbered file *and* its block in `Database/setup-xampp.ps1`, then run the runner. Scope is always the `gate_pass_system` database only.
4. **Verify after every change** — the health endpoint, the login page at `http://192.168.9.7/FormRequestSystem/`, and the specific screen you changed.
5. **Never touch** Apache, MariaDB, Wi-Fi, router, firewall, or the centralized-system folders.

> The server-side handoff log lives in `Docs/server access documentation.md` — every deployment adds a dated entry there, so the server's history can be reviewed without relying on chat logs or memory.

10 System health check and known issues

Checked live over SSH on 18 July 2026. Reference for the next maintenance window — nothing here blocks daily use.

10.1 Live server status — all good

Check	Result
API health probe	Healthy; database Connected
Web app	Loads normally (HTTP 200)
Database migrations	Complete — all versions 001 through 022 applied
Data	95 employees, 96 accounts, 30 requests; in active daily use (latest request filed the same morning)
API build	Compiles with 0 warnings and 0 errors; live copy rebuilt the same day
Apache request forwarding	Configured and working (same-origin API calls, caching, compression)
Backups	Date-stamped backup folders present; latest 13 July 2026

10.2 Observations from live data (operational)

#	Observation	Suggested action
O1	Eight requests filed between 29 June and 9 July are still sitting in a pending-approval status — approvers have not acted on them.	Remind approvers; consider an automatic reminder or escalation after a set number of days (see Section 11).
O2	Three approved passes were never scanned OUT at the gate, and nothing automatically expires them — they stay in the guard's queue indefinitely.	Add a scheduled job that moves stale approved passes to Expired (the status already exists; only the automation is missing).

10.3 Code-level notes

#	Level	Finding
C1	**Medium**	Several approver and scheduling rights are hard-coded to specific Employee IDs in both the frontend and the API (HRAD assignment, calendar access, service-vehicle booking). A personnel change requires a code edit and redeployment instead of a data change. Recommended: move these lists into the existing routing/permission tables.
C2	**Medium**	A testing-only endpoint that permanently hard-deletes a request *including its audit history* is compiled into the live API (guarded by an admin permission, but it contradicts the "nothing is deleted" principle). Recommended: exclude it from production builds.
C3	**Medium**	A pass's QR code never really expires: the server automatically extends the expiry whenever the code is requested, and the code itself is deterministic. A leaked printout stays scannable for the life of the pass. Acceptable on a closed LAN; revisit if the system is ever exposed more widely.
C4	**Low**	One security lookup compares the employment status against `'Active'` while everywhere else uses `'ACTIVE'`. Harmless under the current database collation, but a latent bug if the collation ever changes.
C5	**Low**	Four admin-page pagination helpers reference functions that no longer exist (leftovers from an older admin screen). They are not wired to any button today, but would crash if ever called.
C6	**Low**	One frontend function for the material form is defined twice (the second definition silently wins); two role checks in the navigation code can never match. Dead code — safe, but worth cleaning.
C7	**Low**	Eight debugging log statements remain in the QR scanner code path (visible in the browser console only).
C8	**Low**	Cache-busting version tags in `index.html` are maintained by hand and currently mixed across files. Works, but easy to forget on deployment; a single shared version constant would be safer.
C9	**Low**	Documentation drift: the developer guide still points to an old frontend folder, and the early API contract document differs from the live endpoints. The code is correct; the older documents need refreshing.

11 Expansion roadmap — growing beyond gate passes

The most important architectural fact about this system: **"gate pass" is just the first tenant of a general form-request platform.** Login, roles, approval routing, e-signatures, notifications, printing, QR scanning, and the audit trail are all shared machinery — they were built once and every new form type reuses them.

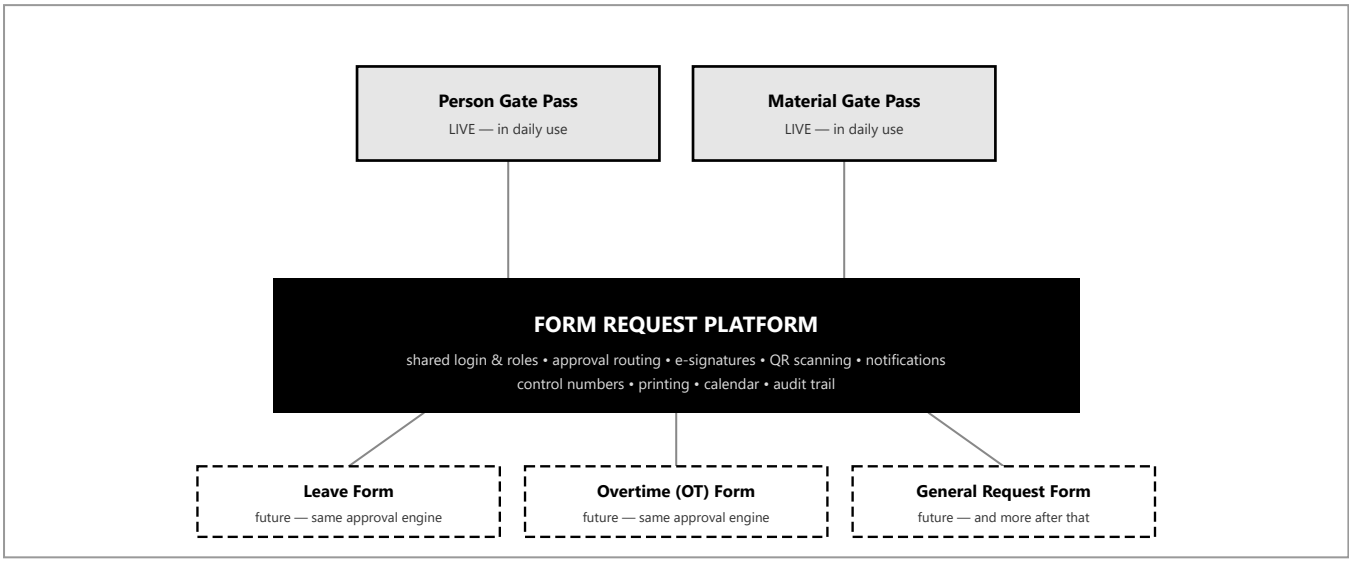

Figure 6 — Two form types live today; the platform underneath is ready for more

11.1 Why adding a form type is cheap here

- **Form types are database rows.** The system already routes, numbers, lists, and audits requests by a form-type code. Person and Material prove the pattern with two very different shapes (a movement pass with QR, and an itemized release list with photos).
- **Approval chains are configuration.** A Leave Form that needs Superior first and then HR is just routing rows in the assignment table — the approval screens, signatures, rejection reasons, and notifications work unchanged.
- **Statuses are extensible.** If a new form needs an extra state, it is a seeded status row with flags, not a schema rewrite.

11.2 The recipe for a new form (using Leave Form as the example)

1. **Seed the form type** — one row (for example `LEAVE_FORM`) in the form-types table.
2. **Add its specific fields** — a small child table (leave dates, leave type, reason) via a numbered migration plus its runner block, exactly like the material-items table was added for Material Gate Pass.
3. **Create the stored procedure** — a create-procedure that validates the fields and reuses the shared control-number generator, mirroring the existing material-pass procedure.
4. **Configure the approval route** — rows in the routing table saying which steps a Leave Form passes through and who acts on each step.
5. **Add the API surface** — a create endpoint and DTO in the API following the existing material-pass endpoint as the template; lists, details, approvals, and notifications come free.
6. **Add the frontend module** — one JavaScript file for the form screen plus a third card on the "New Request" chooser, following `material-gatepass.js` as the template; add a print layout only if the form needs one.
7. **UAT and deploy** — the same sign-off sheet and SSH deployment routine in this document apply.

An **OT Form** follows the identical recipe (fields: date, hours, reason; route: Superior then HR), and a **General Request Form** can start even simpler with free-text fields. Each new form is roughly a one-to-two-week effort because 80% of the machinery already exists.

11.3 Platform improvements worth scheduling alongside

Improvement	Why
Move hard-coded approver IDs into the routing tables	Personnel changes become data edits (this is finding C1 in Section 10 — do it before adding new forms so every form benefits).
Background job for overdue and expiry	Auto-flag overdue returns and expire stale approved passes (finding O2); the same job can send approval-reminder notifications (finding O1).
HTTPS on the office server	Enables phone cameras for scanning everywhere and removes browser warnings; an internal certificate or company domain both work.
Scheduled database backups	Backups currently accompany deployments; a nightly automatic dump protects against everything else.
Email or Viber-style digests	Approvers who rarely open the app still learn that something is waiting for them.

Reports module	Monthly summaries per department, vehicle utilization, and gate traffic — the data is already collected.
Single sign-on with the centralized dashboard	The Form Request tile already lives on the company dashboard; a shared login would remove the second password.

12 Appendix — repository map and common commands

12.1 Repository layout

Path	Contents
`index.html`	The single web page that hosts the entire app.
`Frontend/Functions/`	The live JavaScript modules — one file per feature (gate pass, material, approvals, scanner, calendar, admin, signatures, manual).
`Frontend/Design/`, `Frontend/Connectors/`	Styles and the API client; `runtime-config.js` picks the right API address per environment.
`Backend/`	ASP.NET Core 8 solution — controllers in `Backend/Controllers/`, domain logic in `Backend/Project/`.
`Database/`	`schema.sql`, seeds, stored procedures, numbered `Migrations/`, and the runner `setup-xampp.ps1`.
`Docs/`	Planning documents, API contract, user manuals per role, and the server handoff log.
`LocalData/`	Ignored by Git — real employee data, credentials, and backups stay here, never in the repository.

12.2 Commands used day to day

Command	Purpose
`start-local.bat`	Run the full stack on a developer PC (database, API, frontend).
`deploy-xampp.bat`	Build and copy the app into the local XAMPP for LAN testing.
`Database\setup-xampp.ps1`	Create or migrate the database — safe to re-run any time.
`ssh -i ~/.ssh/id_ed25519_gatepass User@192.168.9.7`	The only approved way into the live server (Section 9).
`curl http://192.168.9.7:5087/api/health`	One-line production health check.

12.3 Related documents

- **UAT Sign-Off Sheet** — the one-page acceptance checklist that accompanies this document.
- **User manuals** (per role, with screenshots) — `Docs/User_Manuals/FormRequest_Documentation/`, also available inside the app under Help.
- **Server handoff log** — `Docs/server access documentation.md` (canonical, update-in-place).
- **API contract and migration plans** — `Docs/API_CONTRACT.md`, `Docs/MIGRATION_PLAN.md`.